

How the Decision and Experimentation Platform Works

One transaction, followed end to end, with the data model behind it. Written to be explained on a whiteboard in fifteen minutes — the last section is the sixty-second version.

Version	1.0
Companion to	Spec v3.5 §17, §18
Audience	Product, engineering, data science
Purpose	Shared vocabulary

Contents

00	The one idea
01	End-to-end flow
02	Worked example
03	Data model
04	Assignment rules
05	What gets recorded
06	Reading the result
07	Naming conventions
08	Three ways this breaks
09	The sixty-second version

The one idea

A Flow is a complete, named, versioned way of processing one class of transaction. A variant Flow differs from the baseline Flow by exactly one strategy. An Experiment splits traffic between them.

That last sentence is the whole design, and the constraint in the middle of it is what makes the rest work. If a variant Flow differs from baseline by exactly one strategy, then any difference in outcome is attributable to that strategy without further argument. Change two things and you have measured a bundle.

Term	What it is	Example
Txn Type	The classification a transaction resolves to. Determines which Flows are eligible	US_ECOM_CREDIT_CIT
Decision Point	A place in the flow where a choice is made	Message augmentation, credential selection, routing, authentication
Strategy	How one decision point decides, for one population	MsgAug_DropCVV_v3
Flow	A complete execution path: one strategy bound at every decision point	Flow_1_BAU_v2, Flow_2_MsgAugDropCVV_v1
Experiment	A traffic split between a baseline Flow and one variant Flow, for one Txn Type	EXP_MsgAugDropCVV_vs_BAU
ExecID	Which Flow this transaction actually ran, recorded on the transaction	Flow_2_MsgAugDropCVV_v1

Why Flow rather than "plan version"

The architecture specification calls this an execution plan with a version number, which is correct and unreadable — exe_01JQ7F3M2Kq42 tells an operator nothing. Flow names carry their own meaning, and the thing you say out loud in a review should be the thing recorded on the transaction. Internally it is still an opaque identifier with a version; the Flow name is the label bound to it.

End-to-end flow



Two placements that matter

Screening sits before assignment because it is never an arm of anything. Constraint binding sits before Plan because the action space must exist before a Flow can be validated against it — a Flow that could select a prohibited action should fail to compile, not fail at runtime. Everything downstream of constraint binding is selecting within a set that is already known to be permitted.

Worked example

One experiment, followed from registration to readout. This is the example to draw on a whiteboard.

The hypothesis

"For US ecommerce credit CIT transactions on BIN range 222333, issuers in that range decline more often when CVV is present in the authorization message. Dropping CVV for that range will improve approval by at least 25 bps without increasing fraud."

What gets registered

STRATEGY	MsgAug_DropCVV_v1
decision_point	message_augmentation
class	DETERMINISTIC
population	txn_type = US_ECOM_CREDIT_CIT
	AND bin_range = 222333
rule	omit CVV field from auth message
fallback	MsgAug_Standard_v7
owner · review_date	
FLOW	Flow_2_MsgAugDropCVV_v1
txn_type	US_ECOM_CREDIT_CIT
baseline_ref	Flow_1_BAU_v2
bindings	identical to baseline at every
	decision point EXCEPT
	message_augmentation → MsgAug_DropCVV_v1
diff_count	1 ← enforced at registration
EXPERIMENT	EXP_MsgAugDropCVV_vs_BAU
txn_type	US_ECOM_CREDIT_CIT
layer	message_augmentation
unit_of_randomisation	payment_intent
arms	90% Flow_1_BAU_v2 (control)
	10% Flow_2_MsgAugDropCVV_v1
primary_metric	approval_rate
guardrails	fraud_proxy · dispute_proxy ·
	per_merchant_harm · latency
sample_size	pre-registered, powered for 25 bps
horizon	approval: hours · fraud: 30d ·
	dispute: 120d
planned_end · hard_expiry	

Two transactions, same merchant, same minute

	Txn_1_A	Txn_1_B
Txn Type	US_ECOM_CREDIT_CIT	US_ECOM_CREDIT_CIT
Eligible	Yes	Yes
Hash bucket	0-89 → control	90-99 → variant
ExecID	Flow_1_BAU_v2	Flow_2_MsgAugDropCVV_v1
Experiment	EXP_MsgAugDropCVV_vs_BAU	EXP_MsgAugDropCVV_vs_BAU
Arm	control	variant
Propensity	0.90	0.10
Strategies applied	MsgAug_Standard_v7, CredSel_Token_v4, Route_LCR_v11	MsgAug_DropCVV_v1, CredSel_Token_v4, Route_LCR_v11
Message	CVV present	CVV omitted

Everything except one field is identical, by construction. That is what makes the comparison mean something, and it is enforced by the `diff_count = 1` check at Flow registration rather than by the discipline of whoever built it.

Data model



Core entities

DecisionPoint	
└ decision_point_id	message_augmentation
└ domain · owner	
└ input_schema	context fields it may read
└ action_schema	shape of its decision
└ constraint_binding	how its action space is computed
└ default_strategy	what runs absent anything else
└ latency_budget_ms	
Strategy	
└ strategy_id · version · owner	
└ decision_point_id	
└ class	DETERMINISTIC PARAMETRIC LEARNED
└ population	predicate over the transaction context
└ definition	rule parameter set policy artefact
└ fallback_strategy_id	required, never implicit
└ objective_ref	for PARAMETRIC and LEARNED only
└ governance	validation record, approver, review date
└ state	DRAFT · BACKTESTED · EVALUATED ·
└ expiry	PILOT · ACTIVE · SUPERSEDED · RETIRED
mandatory	
Flow	
└ flow_id	Flow_2_MsgAugDropCVV_v1
└ txn_type	
└ role	BASELINE VARIANT
└ baseline_ref	required when role = VARIANT
└ bindings[]	{ decision_point_id, strategy_id }
└ diff_count	one entry per decision point, no gaps
└ compiled_at · signature	computed; must equal 1 for a VARIANT
└ state · expiry	
TxnType	
└ txn_type_id	US_ECOM_CREDIT_CIT
└ predicate	channel · funding · region · mandate ·
└ baseline_flow_id	instrument attributes
└ eligible_flows[]	the default; always exactly one

Experiment entities

Experiment	
└ experiment_id · owner	
└ txn_type · layer	layer = the decision point being varied
└ hypothesis	one sentence, written before launch
└ unit_of_randomisation	payment_intent customer credential
└ population	merchant
└ exclusions[]	exposure predicate; may narrow txn_type
└ primary_metric	opted out · strategic accounts ·
└ guardrails[]	holdout · migration soak
	exactly one
	metric + threshold + abort action

```

└─ sample_size · power_for  effect size it can detect
└─ outcome_horizons{}      per metric
└─ state · planned_end · hard_expiry
└─ decision                 PROMOTE | REJECT | INCONCLUSIVE

ExperimentArm
└─ arm_id · experiment_id
└─ flow_id
└─ role                     CONTROL | VARIANT
└─ allocation               shares sum to 1.0 within the experiment

Assignment
└─ assignment_id · experiment_id · arm_id
└─ unit_type · unit_id      what was hashed
└─ bucket · salt            reproducible
└─ propensity               probability this arm was chosen
└─ assigned_at
└─ eligible                 false + reason when excluded

```

Record entities

```

ExecutionRecord          one per payment attempt
└─ attempt_id · intent_id · txn_type
└─ exec_id               the Flow that ran
└─ flow_version
└─ experiment_id · arm_id · assignment_id  (null if not assigned)
└─ strategies_applied[]  { decision_point, strategy_id, version }
└─ artefact_versions{}   constraint_set · bin_snapshot ·
└─                     routing_table · models
└─ constraint_provenance[] which authority produced which bound
└─ timings{}

DecisionRecord            one per decision point per attempt
└─ decision_point_id · strategy_id · version
└─ action_space[]         what was permitted
└─ considered[]           what was evaluated
└─ action_taken
└─ rejected[]             { action, reason_code } ← the
└─                     "why not" view
└─ fallback_would_have_been required, enables attribution
└─ regime                 POLICY | HEURISTIC | LEARNED | FALLBACK
└─ propensity             required when regime = LEARNED
└─ duration_ms

Outcome                   joined asynchronously, several times
└─ attempt_id
└─ approval · auth_code · decline_class
└─ cost{ interchange, scheme, processor }
└─ fraud{ flag, amount, observed_at }
└─ dispute{ flag, amount, observed_at }
└─ horizon_complete[]     which metrics are final

```

Assignment rules

Rule	Why
Assignment happens once, at context assembly, and is carried forward	Retries and later attempts must stay in the same arm, or the arms are not independent
Unit of randomisation is at or above the level the outcome is measured	Randomising retry per attempt measures nothing, because attempt two exists because of what attempt one did
One active experiment per layer per population	Two experiments splitting the same traffic on the same decision point double-allocate it
Independent hash salt per layer	Lets a transaction be in a routing experiment and an authentication experiment at once without correlation
A permanent holdout is excluded from everything	Without it, cumulative platform effect over time is not measurable — only individual experiments are
Opt-out is honoured at assignment, not filtered at analysis	An opted-out merchant is never assigned, rather than assigned and discarded
Guardrail breach aborts the arm automatically	Removing traffic is the action; raising a ticket is not
Transactions in a migration soak slice are excluded	Otherwise a movement cannot be attributed to the arm or to the new execution path

What gets recorded

The transaction carries enough to reconstruct the decision years later, without needing any other artefact to still exist.

```
{
  "attempt_id": "att_01JX...",
  "txn_type": "US_ECOM_CREDIT_CIT",
  "exec_id": "Flow_2_MsgAugDropCVV_v1",

  "experiment": {
    "experiment_id": "EXP_MsgAugDropCVV_vs_BAU",
    "arm": "variant",
    "assignment_id": "asg...",
    "unit_type": "payment_intent",
    "propensity": 0.10
  },

  "strategies_applied": [
    { "decision_point": "message_augmentation",
      "strategy": "MsgAug_DropCVV_v1", "regime": "POLICY" },
    { "decision_point": "credential_selection",
      "strategy": "CredSel-Token_v4", "regime": "LEARNED",
      "propensity": 0.72 },
    { "decision_point": "routing",
      "strategy": "Route_LCR_v11", "regime": "LEARNED",
      "propensity": 0.64 }
  ],

  "artefact_versions": {
    "constraint_set": "v311", "bin_snapshot": "v52",
    "routing_table": "v184", "risk_model": "fraud_v19"
  },

  "outcome": {
    "approval": true, "auth_code": "00",
    "cost_bps": 174,
    "fraud": { "status": "pending", "horizon_days": 30 },
    "dispute": { "status": "pending", "horizon_days": 120 }
  }
}
```

The two fields people leave out

Propensity and the fallback action. Without propensity, off-policy evaluation is impossible and the platform can only learn from live experiments, which is the slow path. Without the fallback action recorded on every decision, a strategy's contribution cannot be attributed against anything. Both are cheap to log and expensive to retrofit, because you cannot backfill them for traffic that has already run.

Reading the result

EXP_MsgAugDropCVV_vs_BAU		status: READOUT	
population: US_ECOM_CREDIT_CIT, BIN 222333			
n = 1,240,000 intents		control 90% · variant 10%	
PRIMARY			
approval rate	control 87.31%	variant 87.68%	
	delta +37 bps	CI [+18, +56]	passes
GUARDRAILS			
fraud proxy (30d)	+0.4 bps	CI [-1.1, +1.9]	within
dispute proxy	pending, 120d horizon		monitored
per-merchant harm	none beyond CI at minimum volume		passes
latency	no change		passes
ECONOMICS			
approval value	+\$XX per \$1M processed		
cost delta	none – no routing change involved		
net	positive, driven entirely by approval		
DECISION		PROMOTE to 50%, full ramp on 120d dispute readout. Strategy becomes the default binding for this population; Flow_1_BAU_v3 is compiled with it, and the experiment closes.	

Promotion is what turns a variant into the new baseline: a new baseline Flow version is compiled with the strategy bound, and the next experiment on this Txn Type measures against that. This is how the baseline ratchets rather than drifting.

Naming conventions

Names are the interface people actually use. These are worth fixing before there are two hundred of anything.

TXN TYPE	REGION_CHANNEL_FUNDING_MANDATE US_ECOM_CREDIT_CIT · US_CP_DEBIT_PIN · EU_ECOM_CREDIT_MIT
STRATEGY	DecisionPoint_WhatItDoes_vN MsgAug_DropCVV_v1 · CredSel_PreferToken_v4 Route_LeastCost_v11 · Auth_ExemptTRA_v2 Never Strategy1, Strategy2 – the number tells you nothing six months later and the name is what appears in the readout, the ticket and the incident
FLOW	Flow_N_Descriptor_vN Flow_1_BAU_v2 · Flow_2_MsgAugDropCVV_v1 A variant Flow is named for the one thing that differs from baseline, because that is all that differs from baseline
EXPERIMENT	EXP_Variant_vs_Baseline EXP_MsgAugDropCVV_vs_BAU Reads as the question being asked

Three ways this breaks

1 – Two experiments on the same traffic

If `EXP_A` allocates `Txn_Type_1` at 90/10 and `EXP_B` also allocates `Txn_Type_1` at 90/10 on the same decision point, the transaction is in both and the allocations do not compose. Either the populations must be disjoint, or the two experiments must sit on different layers with independent salts so that assignment is genuinely orthogonal. The registry rejects the second registration and names the conflict; it is not caught in analysis, because by analysis time the data is already unusable.

2 – Variant flows that differ by more than one thing

The most natural mistake: a variant Flow that changes message augmentation *and* credential selection, because both were ready at the same time. The experiment then measures a bundle, and if it wins you cannot tell which half to keep — or whether one half is negative and being carried by the other. The `diff_count = 1` check at Flow registration is the only thing preventing this, and it should be enforced by tooling rather than by review.

3 – Flows and strategies that never expire

Every Flow, Strategy and Experiment carries a mandatory expiry. Without it, the registry fills with variants nobody will defend and nobody will delete, strategies that never fire because a higher-authority one always precedes them, and experiments that were never formally concluded. Inert strategies are reported as inert, expired experiments close as INCONCLUSIVE, and unreviewed strategies are suspended rather than silently continued.

The sixty-second version

For the meeting where someone asks what the decision platform actually does.

Every transaction gets classified into a type.

Each type has a baseline Flow – a complete, named way of processing it, with one strategy bound at every decision point: how we format the message, which credential we use, where we route it, whether we authenticate.

When we have an idea, we compile a variant Flow that is identical to the baseline except for one strategy, and we run an experiment: ninety percent baseline, ten percent variant, split on a stable hash so a customer sees the same behaviour on every retry.

Every transaction records which Flow it ran, which experiment it was in, which strategies fired, and – for the learned ones – the probability we would have chosen that action. Outcomes join back over hours, days and months depending on the metric.

When the variant wins on the primary metric without breaching a guardrail, the strategy becomes the new baseline binding and the baseline version increments. When it loses, we have learned something for the cost of ten percent of one population.

The difference between this and configuring a gateway is that every change is measured against what would otherwise have happened, and the platform gets better at deciding whether or not anyone is watching.